

Concepts: FDA behavior-clone system ("parallel")

Every concept used in the build, with the exact code. Project: a five-stage pipeline (Sense Clone Surface) that tracks FDA regulatory changes and predicts stakeholder behavior with three behavior-pattern clones (enforcement, reviewer, sponsor), built on parallel.ai's API suite.

Repo: `shripathmit/parallel`. Live: FastAPI on Railway, auto-deployed from GitHub.

1. LLM-powered web search (Parallel Search) Sense

Concept. Fetch a URL and extract exactly the fields you name, returned as structured data. Turns unstructured FDA pages into machine-readable records (title, date, cited CFR parts, affected product codes).

```
```python  
def extract(self, url, objective):
 return self.post("/v1beta/extract", {
 "url": url,
 "objective": objective,
 })
```
```

3. Agentic task execution with output contracts (Parallel Task) the model must return parseable, typed data.

```
```python  
def task_run(self, objective, output_schema=None,
webhook_url=None):
 body = {"objective": objective}
 if output_schema:
 body["output_schema"] = output_schema
 if webhook_url:
 body["webhook_url"] = webhook_url
 return self.post("/v1/tasks/runs", body)

def wait_for_task(self, run_id, timeout=600, interval=10):
```

```
__# poll task result(run_id) until terminal state or timeout
__
```

~~Every analysis is validated against `CHANGE\_ANALYSIS\_SCHEMA`  
(see concept 12).~~

~~\_\_~~

~~## 4. Chat completions with system prompts (Parallel Chat)  
corpus building~~

~~\*\*Concept.\*\* "Find me all entities matching these criteria"  
continuous sensing~~

~~\*\*Concept.\*\* Declarative monitors: "watch this URL/topic, hit my  
webhook when it  
changes." Push instead of poll. The webhook receiver diffs the  
new capture against  
the stored baseline and emits a change event.~~

```
__python
def create_monitor(self, name, objective, url, frequency,
webhook_url):
__ return self.post("/v1alpha/monitors", {
__ "name": name,
__ "objective": objective,
__ "url": url,
__ "frequency": frequency, # TODO: verify accepted
values
__ "webhook_url": webhook_url, # TODO: verify signing
scheme
__ })
__
__
```

~~\_\_~~

~~## 7. Batched task execution (Parallel Task Groups) Task  
(distill):~~

```
__python
def mine_patterns(client, stakeholder, seed_query=None,
max_entities=25):
__ query = seed_query or SEED_QUERIES[stakeholder]
__ found = client.findall(
__ objective=f"Find FDA records showing {stakeholder}"
behavior patterns: {query}",
__ criteria={"stakeholder": stakeholder, "topic": query,
```

```

 "max_entities": max_entities},
)
 entities = found.get("entities", found.get("results", [])) or []
 run = client.task_run(
 objective=(f"Distill recurring {stakeholder} behavior
patterns from these "
 f"FDA records. Each pattern needs a name, the
trigger "
 f"conditions, and evidence citations.\n\
nRecords:\n"
 f"{json.dumps(entities)[:15000]}"),
 output_schema={"type": "object", "required":
["patterns"],
 "properties": {"patterns": {"type":
"array",
 "items":
BEHAVIOR_PATTERN_SCHEMA}}},
)
 output = client.wait_for_task(run.get("run_id",
run.get("id")))
 return output.get("patterns", []) if isinstance(output, dict)
else []
'''

```

Mined patterns persist into the library so future clones reason from them:

```

'''python
def mine_and_store(client, data_dir, stakeholder):
 mined = mine_patterns(client, stakeholder)
 store = PatternStore(data_dir)
 for m in mined:
 store.add(Pattern(name=m.get("pattern_name",
f"{stakeholder}-pattern"),
 stakeholder=m.get("stakeholder",
stakeholder),
 description=m.get("description", ""),
 evidence_citations=m.get("evidence_citations", []),
 confidence=float(m.get("confidence",
0.5)),
 support=1, demo=False))
'''

```

~~## 9. Deterministic pattern matching each fired pattern reports its score, matched triggers, and confidence.~~

```
```python
def match_patterns(analysis, patterns, stakeholder, top_k=3):
    text = " ".join([
        analysis["summary"],
        analysis["change_type"],
        " ".join(analysis["affected_product_codes"]),
        " ".join(analysis["affected_submission_types"]),
    ]).lower()
    scored = []
    for p in patterns:
        if p.stakeholder != stakeholder or not p.triggers:
            continue
        matched = [t for t in p.triggers if t.lower() in text]
        if not matched:
            continue
        score = (len(matched) / len(p.triggers)) * p.confidence
        scored.append((p, round(score, 3), matched))
    scored.sort(key=lambda s: s[1], reverse=True)
    return scored[:top_k]
```
```

~~Example pattern from the library:~~

```
```python
Pattern(
    name="inspection-refusal-escalation",
    stakeholder="enforcement",
    triggers=["501(j)", "refused inspection", "denied inspection",
              "delayed inspection", "refusal"],
    description="When a firm delays, denies, or refuses FDA inspection, CDER deems "
                "the drugs adulterated under 501(j) and escalates faster toward "
                "import alert or injunction than for CGMP findings alone.",
    evidence_citations=["https://www.fda.gov/.../warning-letters/"
                       "new-life-pharma-llc-725661-04142026"],
    confidence=0.9, support=12, demo=True,
)
```
```

---

## ~~## 10. Evidence-grounded prompting demo-mode architecture~~

~~**Concept.** The system must be fully demonstrable with no API key. On startup, if no key is configured and the store is empty, it seeds 8 demo events built from real FDA publications (real titles/URLs/dates; synthetic analyses labeled `demo: true`). The UI renders DEMO DATA callouts, and "Run analysis/simulation" flows work end to end against canned payloads.~~

```
```python
@app.on_event("startup")
def startup():
    if not settings.parallel_api_key and not store.list():
        seed_demo(settings.data_dir) # real publications,
        synthetic analyses
```
```

---

## ~~## 14. Thin API client (stdlib only)~~

~~**Concept.** One `urllib`-based client wraps every Parallel endpoint (Search, Extract, Task, Chat, FindAll, Monitor, Task Groups) behind typed methods. No SDK dependency, no heavy frameworks. Uncertain API fields are isolated here with defensive parsing and TODO markers, so verifying against real docs touches one file.~~

---

## ~~## 15. Deployment: FastAPI + Railway + GitHub auto-deploy~~

~~**Concept.** Standard 12-factor web service: `Procfile` runs uvicorn, config from environment (`PARALLEL\_API\_KEY`, `DATA\_DIR`, `WEBHOOK\_BASE\_URL`), persistent volume for the JSON event store, Railway redeploys on every push to `main`.~~

---

## ## Architecture recap

```\n\n

Sense (Search/Extract/Monitor) Understand (Task +
CHANGE_ANALYSIS_SCHEMA) Clone (FindAll + Task predictions)
— Surface (FastAPI dashboard: overview, events, clones,
monitors)

```\n\n

The key separation: \*\*mining\*\* (batch, expensive, LLM) vs.  
\*\*matching\*\*  
(deterministic, free, explainable) vs. \*\*reasoning\*\* (LLM,  
grounded in matched  
evidence). That is what makes the clones more than personas in a  
prompt.